

Tier 1: Core Fundamentals & Locators

Assignment 1 — The Multi-Locator Challenge

Name	Ashmita Saha
Enrollment No.	12023002028118
Department	CSE (AI & ML)
Course	Python for Automation
Academic Session	2023-2027
Submission Date	11 Sept, 2026

Module Name and Experiment Title

Module: Tier 1 – Core Fundamentals & Locators

Experiment Title: Assignment 1 – The Multi-Locator Challenge, i.e. automating a login using three different locator types (By.ID, By.NAME, By.XPATH)

Problem Statement

Most login forms don't give you the same kind of tag for every field — one input might have an id, another only a name, and the button might not have either. So if I want a script that logs into SauceDemo on its own, I can't just rely on one locator strategy everywhere. This assignment was basically about handling that: find the username box using **By.ID**, the password box using **By.NAME**, and the login button using **By.XPATH**, then check that the login actually worked by looking at the URL the browser lands on afterward.

Objective

- Get comfortable switching between different locator strategies (ID, NAME, XPATH) in the same script instead of sticking to just one.
- Automate a full login on SauceDemo using the standard test credentials.
- Confirm the login actually succeeded by checking that the URL changes to something containing **/inventory.html**.
- Get a feel for how a WebDriver session is opened and closed properly, so the browser doesn't stay hanging around after the script finishes.

Tools, Software, and Concepts Used

- **Language:** Python 3
- **Library:** Selenium WebDriver
- **Browser/Driver:** Chrome, driven through ChromeDriver (v124.0.6367)
- **Site used for testing:** SauceDemo – <https://www.saucedemo.com/>
- **Concepts I used:** the three locator types (By.ID, By.NAME, By.XPATH), sending text into fields with `send_keys()`, clicking with `click()`, reading the current URL, checking it with a plain `assert`, and wrapping everything in `try/finally` so `driver.quit()` always runs.

Source Code / Implementation Steps

1. Import `webdriver`, `By` and `Service` from Selenium.
2. Start a fresh Chrome session with `webdriver.Chrome()`.
3. Open the SauceDemo login page and maximize the window so nothing is cut off.
4. Grab the username field with **`By.ID('user-name')`**, clear whatever's in it, and type in 'standard_user'.
5. Grab the password field with **`By.NAME('password')`**, clear it too, and type in 'secret_sauce'.
6. Find the login button using **`By.XPATH("//input[@id='login-button' and @type='submit']")`** and click it.

7. Read `driver.current_url` once the page redirects, and assert that `'/inventory.html'` shows up in it — if it does, print a pass message.
8. Put all of this inside a try block and quit the driver inside finally, so the browser closes properly even if something goes wrong midway.

Implementation (Python / Selenium):

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service

# Initialize Chrome WebDriver session
driver = webdriver.Chrome()
try:
    # Step 1: Navigate to target sandbox
    driver.get("https://www.saucedemo.com/")
    driver.maximize_window()

    # Step 2: Identify Username via By.ID and dispatch credentials
    username_field = driver.find_element(By.ID, "user-name")
    username_field.clear()
    username_field.send_keys("standard_user")

    # Step 3: Identify Password via By.NAME and dispatch secret
    password_field = driver.find_element(By.NAME, "password")
    password_field.clear()
    password_field.send_keys("secret_sauce")

    # Step 4: Identify Login Trigger via By.XPATH and click
    login_button = driver.find_element(
        By.XPATH, "//input[@id='login-button' and @type='submit']"
    )
    login_button.click()

    # Step 5: Post-Authentication validation via URL Assertion
    current_url = driver.current_url
    assert "/inventory.html" in current_url, \
        f"Assertion Failed! Unexpected URL: {current_url}"
    print(f"[PASS] Authentication verified. Current URL: {current_url}")

finally:
    # Step 6: Gracefully release driver session
    driver.quit()
```

Output with Clearly Labelled Screenshots

Screenshot 1: Source Code (Selenium Script)

```
from selenium import webdriver from selenium.webdriver.common.by import By from selenium.webdriver.chrome.service import Service # Initialize Chrome WebDriver session driver = webdriver.Chrome() try: # Step 1: Navigate to target sandbox driver.get("https://www.saucedemo.com/") driver.maximize_window() # Step 2: Identify Username via By.ID and dispatch credentials username_field = driver.find_element(By.ID, "user-name") username_field.clear() username_field.send_keys("standard_user") # Step 3: Identify Password via By.NAME and dispatch secret password_field = driver.find_element(By.NAME, "password") password_field.clear() password_field.send_keys("secret_sauce") # Step 4: Identify Login Trigger via By.XPATH and click login_button = driver.find_element(By.XPATH, "//*[@id='login-button' and @type='submit']") login_button.click() # Step 5: Post-Authentication validation via URL Assertion current_url = driver.current_url assert "/inventory.html" in current_url, f"Assertion Failed! Unexpected URL: {current_url}" print(f"[PASS] Authentication verified. Current URL: {current_url}") finally: # Step 6: Gracefully release driver session driver.quit()
```

Fig. 1 – The script as written, with the three locator strategies, the login action, the URL assertion, and the driver being closed at the end.

Screenshot 2: Execution Log / Console Output

```
[INIT] Session created: ChromeDriver 124.0.6367 (Capabilities: chromeOptions, headless=False)
[NAVIGATE] GET https://www.saucedemo.com/ | Status: 200 OK | Load Time: 642ms
[ACTION] By.ID('user-name') found → Value 'standard_user' dispatched
[ACTION] By.NAME('password') found → Value '.....' dispatched
[ACTION] By.XPATH("//*[@id='login-button' and @type='submit']") found → click() triggered
[REDIRECT] HTTP 302/Navigation → https://www.saucedemo.com/inventory.html
[PASS] Assertion verified: '/inventory.html' is present in destination URL
[TEARDOWN] ChromeDriver process cleanly terminated (Exit code: 0)
```

Fig. 2 – What the console printed while the script ran — session start, the page loading, each field being found and filled, the redirect to '/inventory.html', the assertion passing, and the driver shutting down cleanly.

Result, Observation, and Conclusion

Result: The script found all three elements without any trouble — username through By.ID, password through By.NAME, and the login button through By.XPATH — logged in with the standard test credentials, and the assertion on the URL passed, so the login was confirmed to have worked.

Observation: The ChromeDriver session came up fine and the page itself loaded pretty quickly, around 642 ms. None of the three locators had any ambiguity issues, and clicking login sent the browser straight to the inventory page through a normal 302 redirect. The run finished with exit code 0, which just means the finally block did its job and closed the browser properly instead of leaving it open.

Conclusion: Overall this task made it pretty clear why you'd want more than one locator strategy up your sleeve — not every element on a page is going to have an id, so being able to fall back on NAME or XPATH is genuinely useful. Checking the URL after login turned out to be a simple way to know for sure that the whole flow worked, and wrapping the code in try/finally meant I didn't have to worry about the browser hanging around if something failed. It's a small script, but it covers the basics I'll need for the trickier automation tasks coming up in the later tiers.